

>_ Openclaw Channel

Author: Kan Deng, Phd

Date: April 29, 2026

Play Scene

- Ammo_box_3
- Ammo_box_2
- Ammo_box_1
- AmmoTable
- Canvas
- GameUIManager
- =====DEBUG=====
- DebugManager
- =====
- Lighting
- EventSystem
- =====Managers=====
- SpawnInGameManager
- ZoneManager
- ChatManager
- =====Env=====
- map
- NavMesh Surface
- Testing
- PlayerCamera(Clone)
- PlayerFollowCamera(Clone)
- Player(Clone) Local
- PlayerCanvas(Clone)
- Bot#0203
- Bot#9829
- Bot#5563
- Bot#6711
- Behavior Manager
- DontDestroyOnLoad

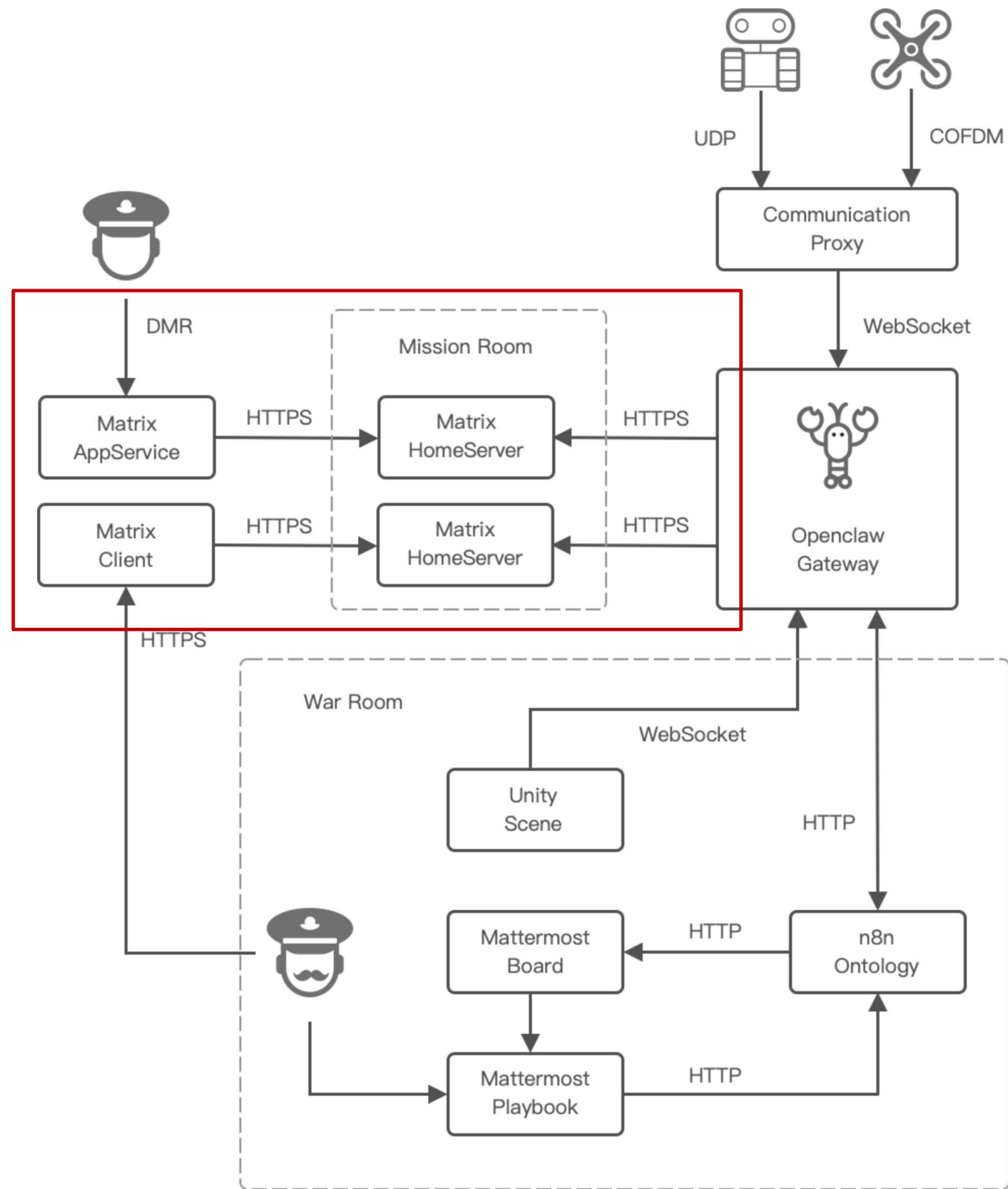


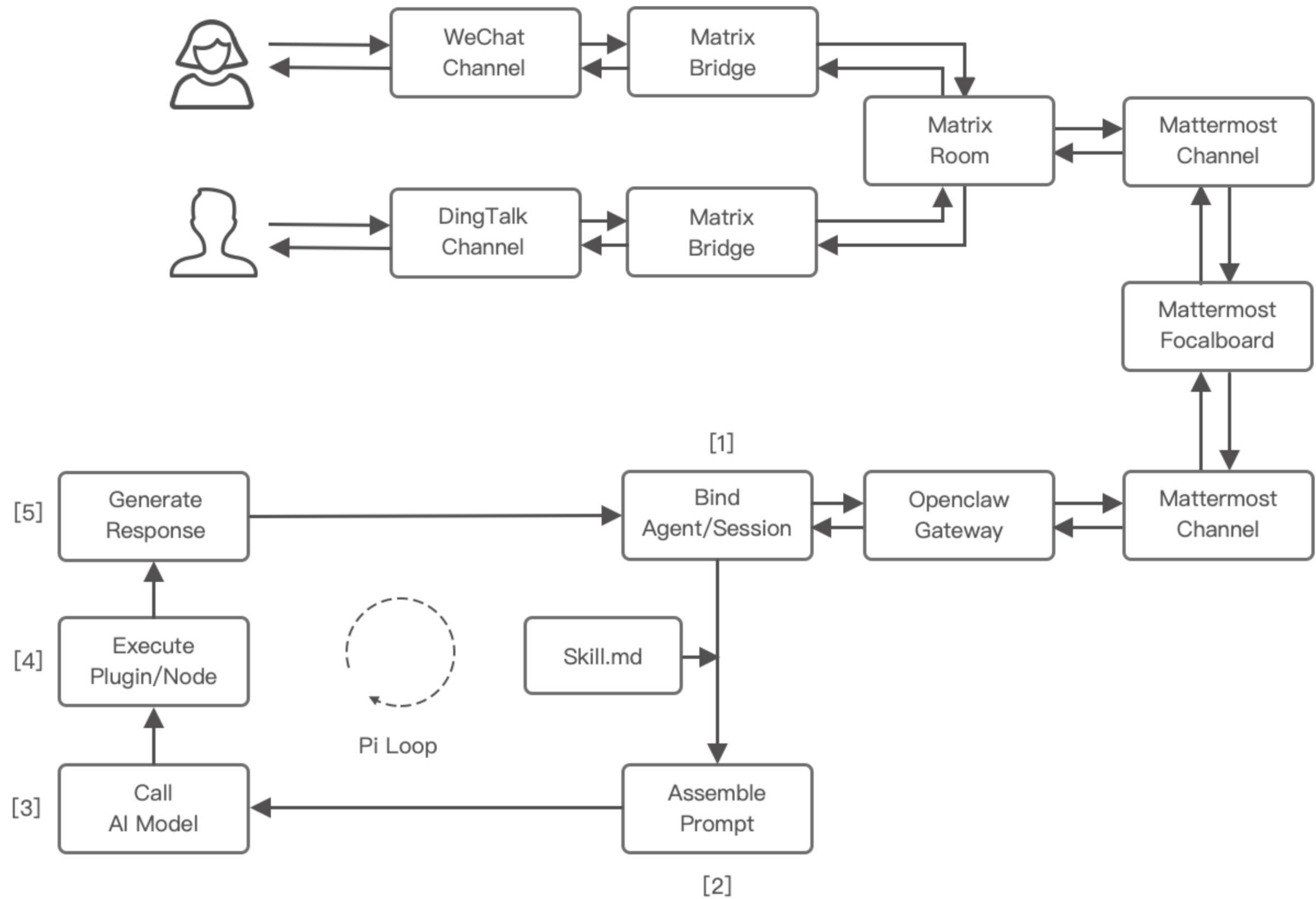
[21:48:34] [Netcode] [Server-Side] Pending Client-0 connection approved!
UnityEngine.Debug:Log (object)

[21:48:34] [PlayerNetwork] Set player name (direct fallback): Player_0
UnityEngine.Debug:Log (object)

[21:48:34] [HandleSpawnBot] Spawning 4 bots
UnityEngine.Debug:Log (object)

[21:48:34] SpawnBot0: NetworkMessage_Tcp_Done: HasNetworkTransform_Tcp

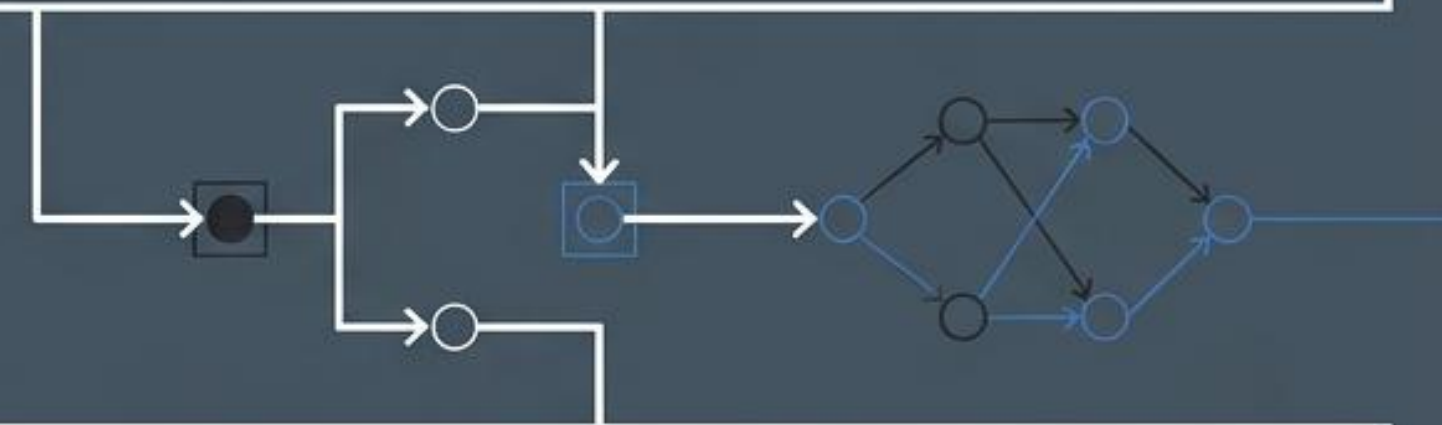




Slack
Discord
Teams

REDEFINING THE MESSAGING PARADIGM

Matrix is not just a messaging app. It is a distributed, eventually consistent, persistent messaging database with an HTTP API.



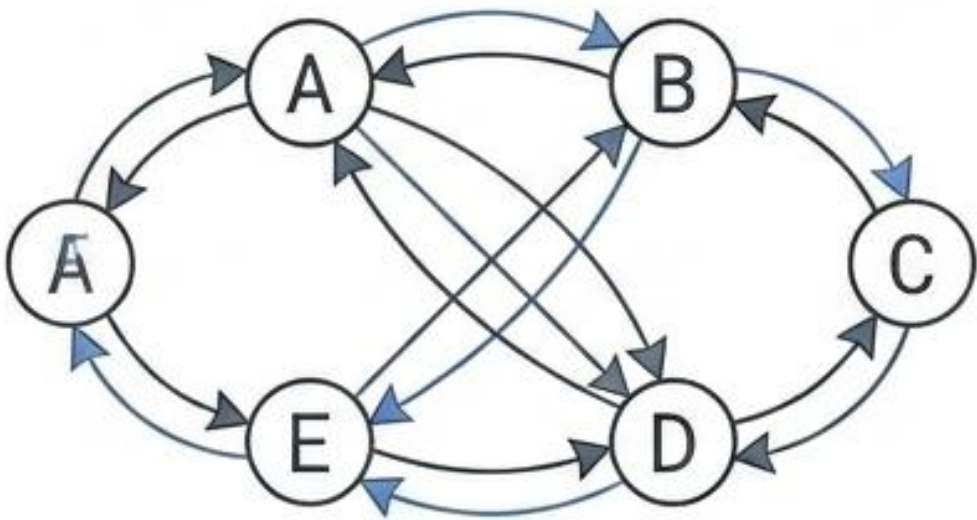
KEY TAKEAWAY: History and group communications are first-class citizens, not ephemeral byproducts of message routing.

Pure Message Passing vs. Replicated History

Traditional (XMPP/Email)

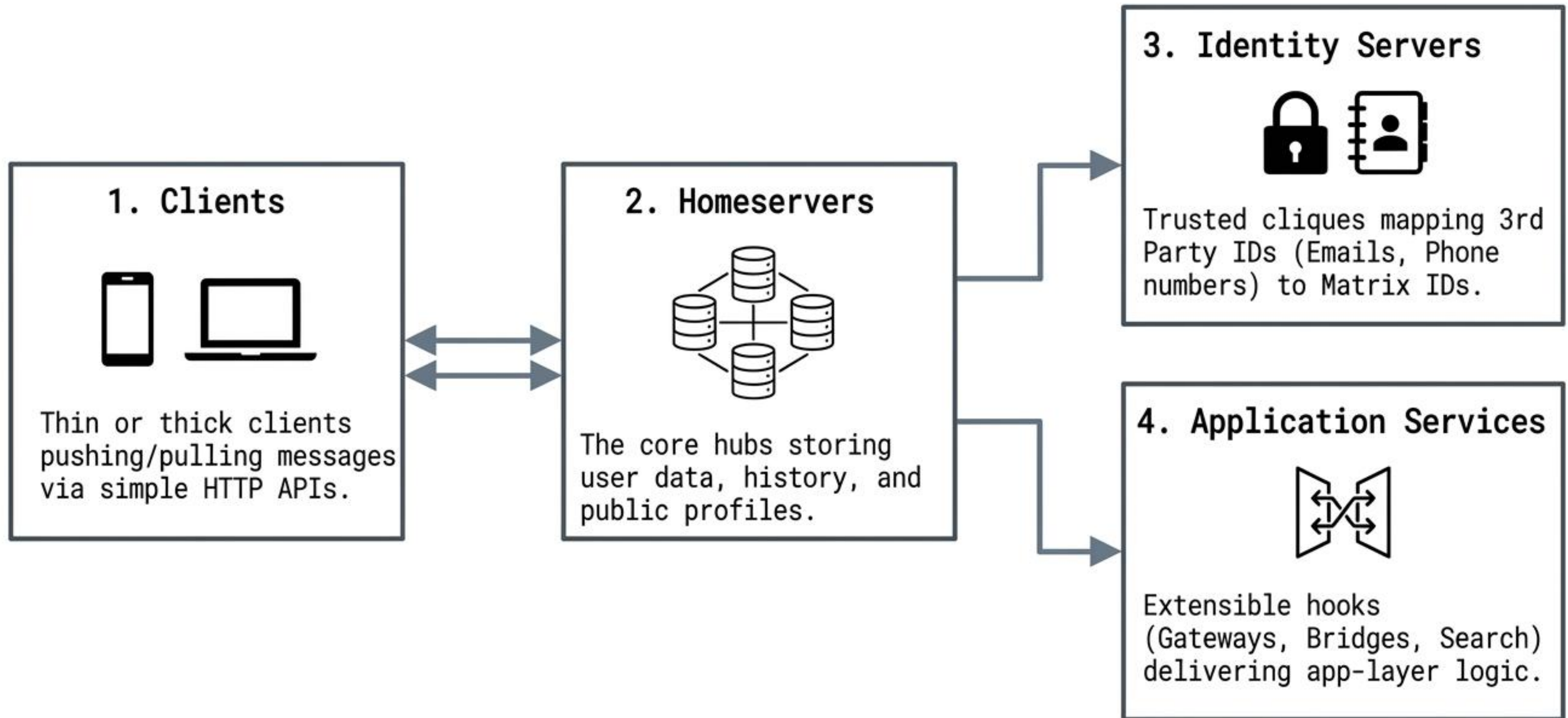


Matrix

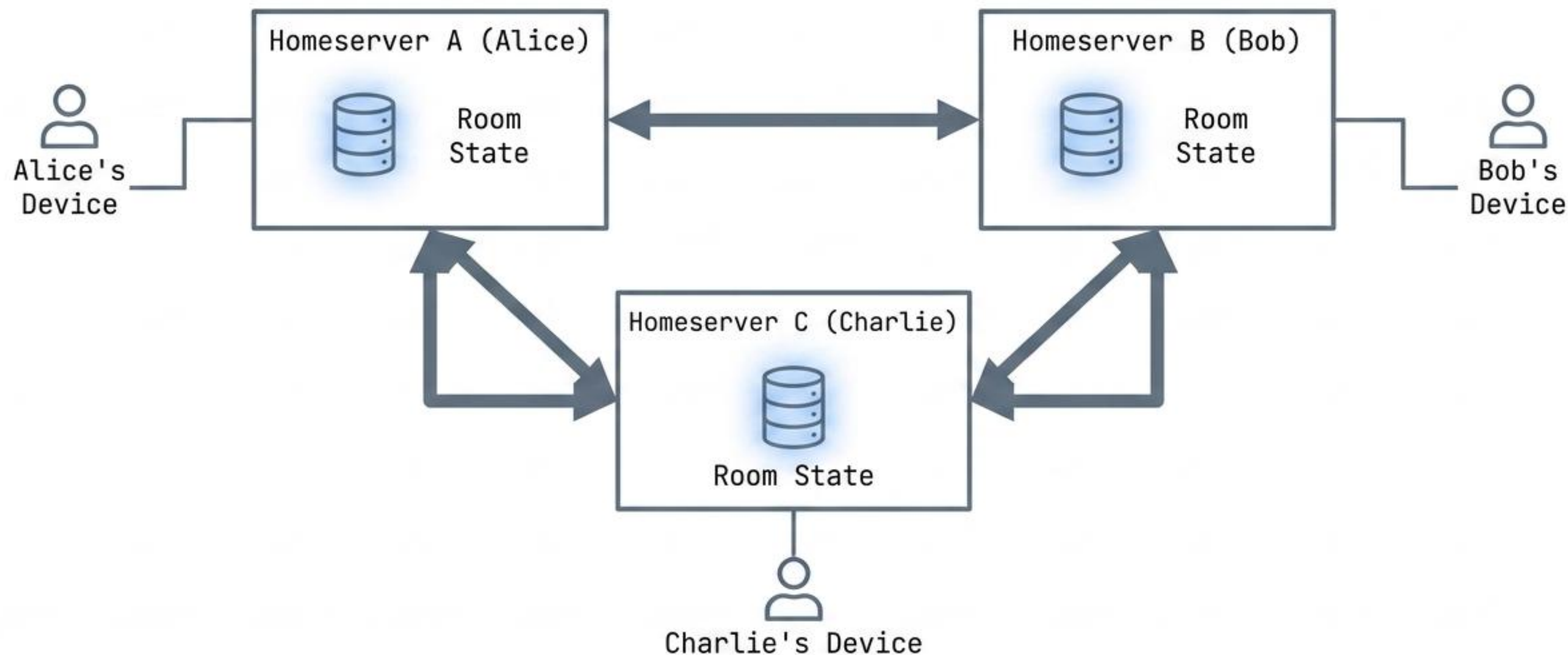


Architecture	Pure Message Passing	Replicated per-topic data structure
State Ownership	Server-owned, fragmented state	Shared state, replicated across all participants
Consistency	Ephemeral, prone to split-brain	Strong Eventual Consistency (SEC)
Resilience	Vulnerable to partition loss	Highly partition-tolerant (CRDT)

The Macro Topology: Four Pillars

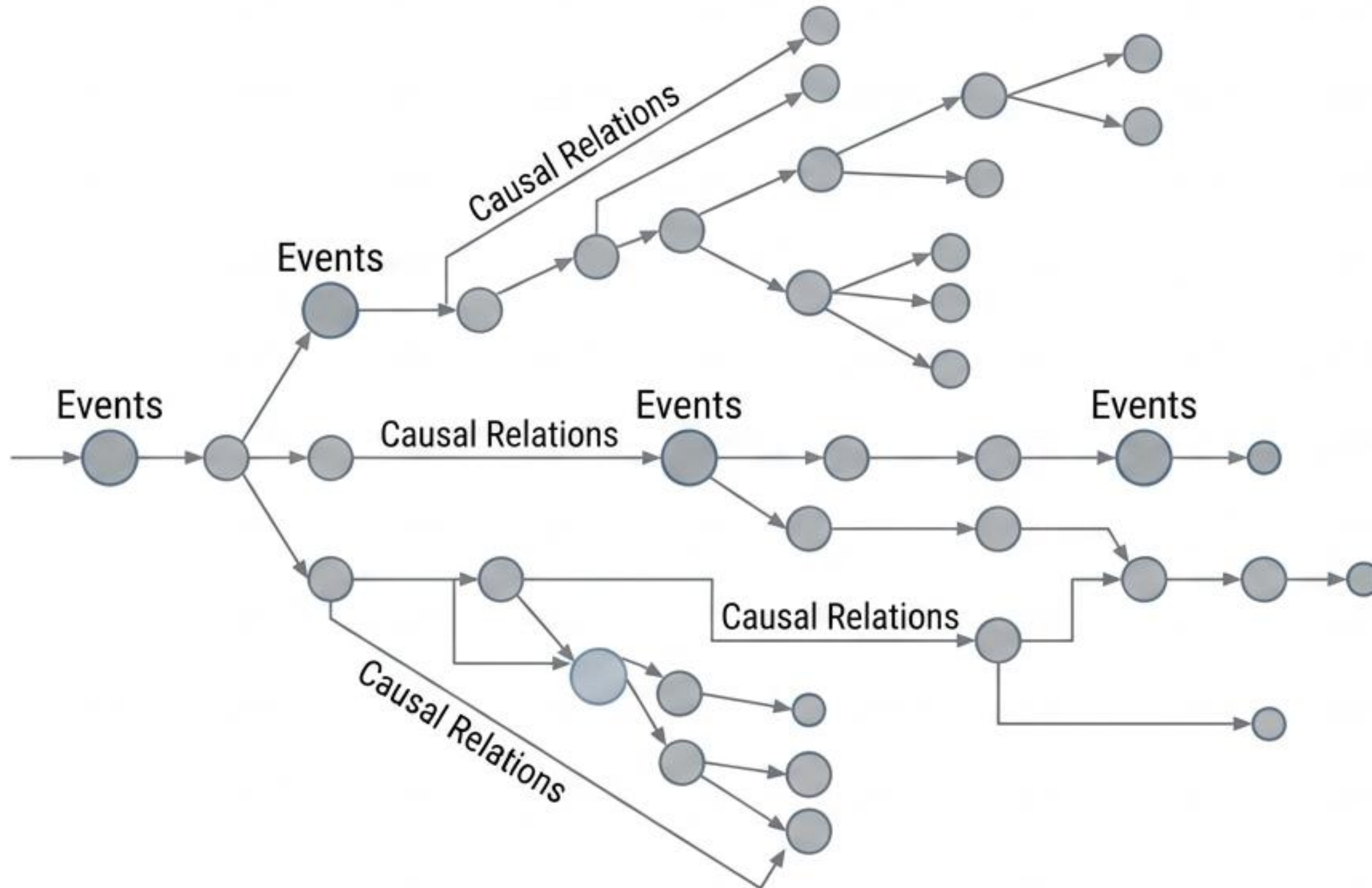


The Federated Room Model



Room data is replicated across all homeservers whose users are participating in a given room. No single homeserver has total control or ownership. If Homeserver A goes down, the room survives on B and C.

The Core Engine: The Matrix Event Graph (MEG)

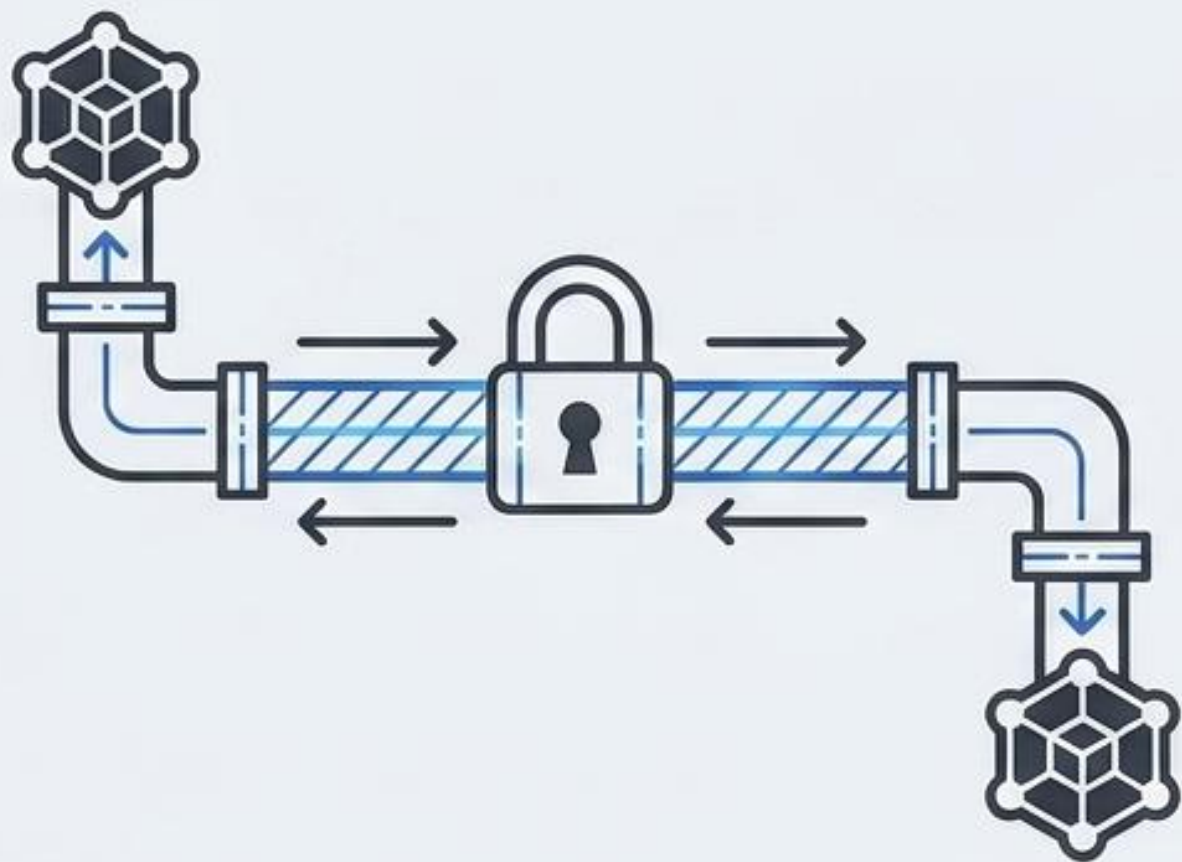


Every room is a DAG.
It is a Conflict-Free
Replicated Data Type
(CRDT).
Appending new events
is the only write
operation supported.

Key Insight: Because
servers synchronize full
causal histories, missing
messages are
algorithmically identified
and backfilled via
targeted server queries.

Securing the Graph: End-to-End Encryption

01m Protocol



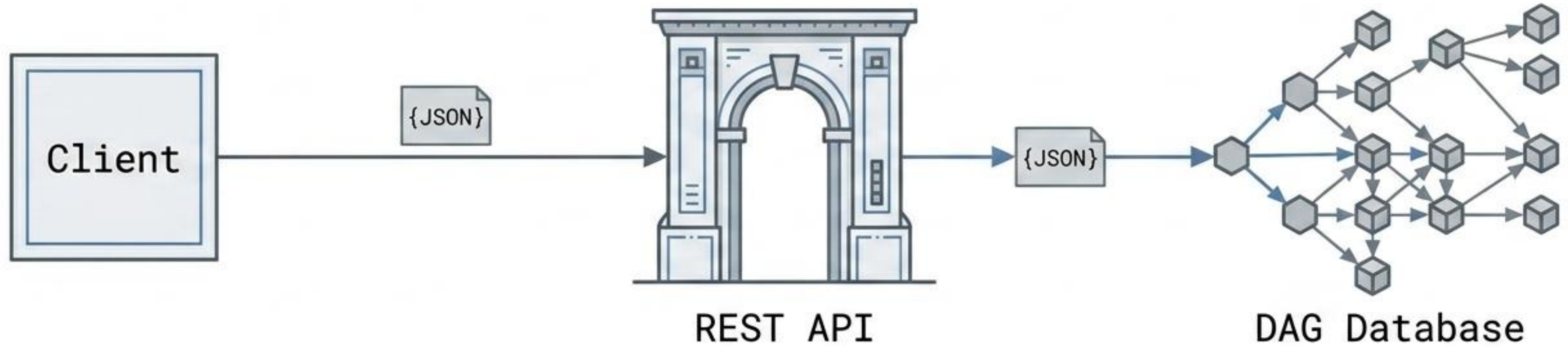
Implements peer-to-peer encryption (Double Ratchet, comparable to Signal). Secures 1:1 communications and key exchanges.

Megolm Protocol



Implements peer-to-multipeer encryption (Sender Keys). Efficiently secures massive group chats without encrypting payloads N times.

The Interface Layer: JSON over HTTP



Matrix abstracts the complex DAG mathematics behind pragmatic, web-friendly APIs.

The Event Primitives:

Everything pushed to the graph is a JSON event. Whether it is an instant message, a WebRTC call setup, IoT sensor data, or typing indicators—it is all just synchronized JSON payloads.

Anatomy of Extensible Payloads

m.room.message

```
{  
  "msgtype": "m.text",  
  "body": "hello"  
}
```

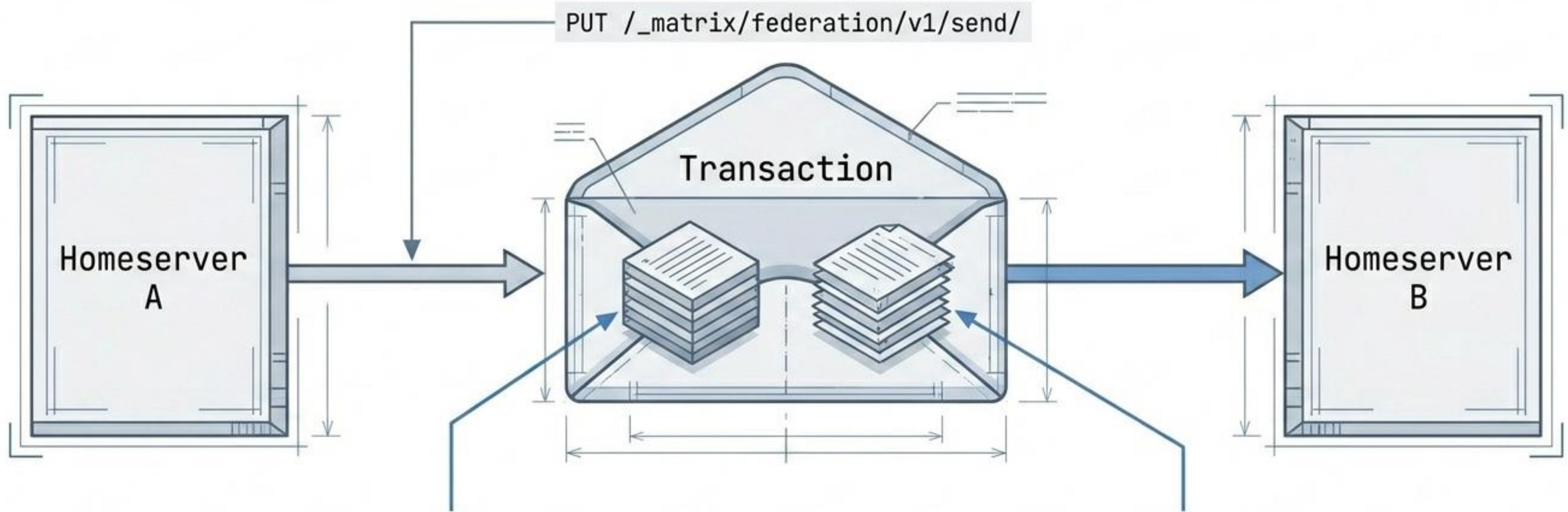
org.matrix.demos.unity.stickmen

```
{  
  "actions": [  
    {"x": "0.552", "y": "6.224", "t": "0.947"},  
    {"x": "0.551", "y": "6.220", "t": "0.970"}  
  ]  
}
```



The homeserver does not care what the JSON contains. It persists and synchronizes the payload across the federation with cryptographically secure eventual consistency.

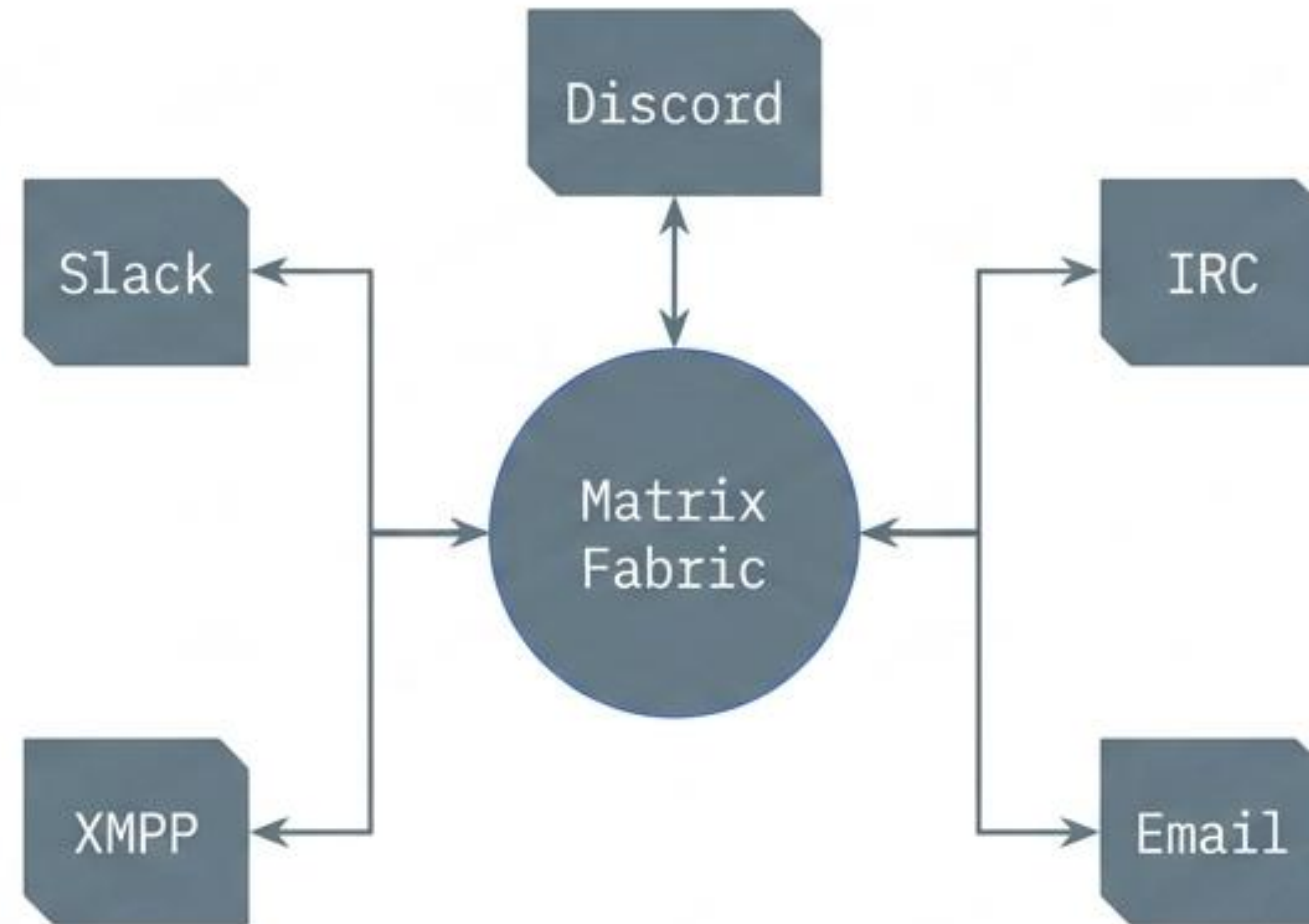
Federation in Action (Server-to-Server)



PDUs (Persistent Data Units)
Room events that become permanent nodes in the Event Graph (messages, state changes).

EDUs (Ephemeral Data Units)
Live activity that is NOT persisted in the graph (typing notifications, read receipts, presence).

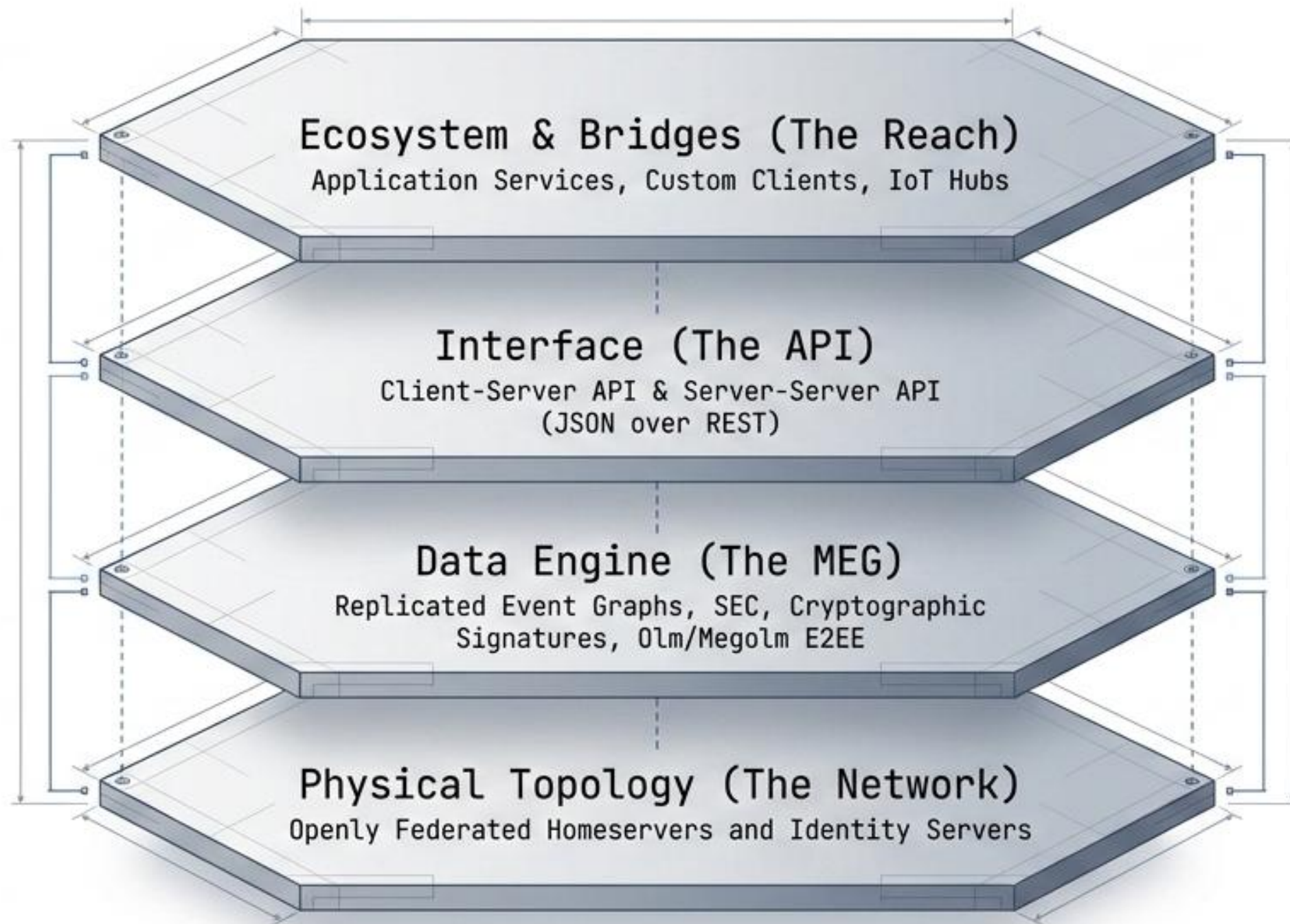
Bridging the Silos: Application Services



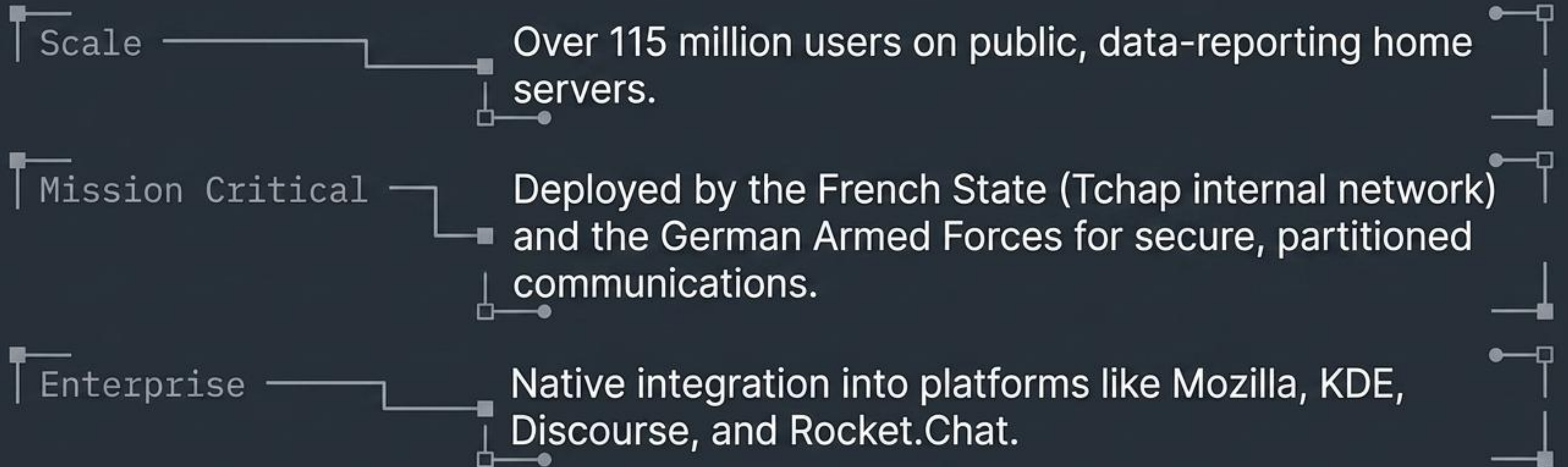
Mechanism: The Application Service API (AS API) allows custom server-side behavior.

Function: Bridges act as puppet users on external networks, translating proprietary APIs into standard Matrix JSON events and injecting them into the decentralized DAG.

Synthesis: The Matrix Architecture Stack



Proven at Scale



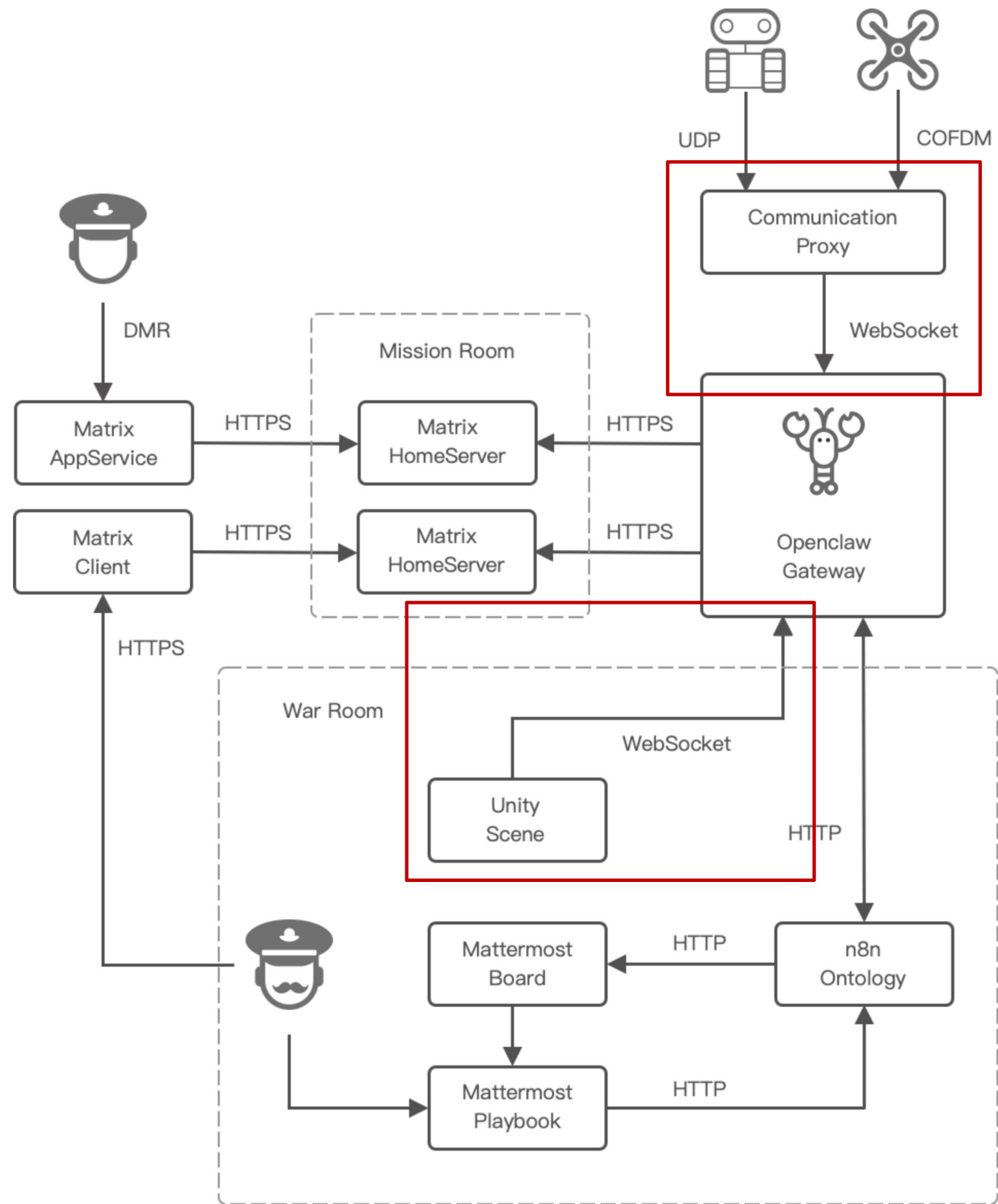
A battle-tested blueprint for the future of decentralized data.

[**matrix**]

>_ Openclaw Channel

Author: Kan Deng, Phd

Date: April 29, 2026





OpenClaw

The Local Brain

Runs the Gateway. Manages persistent AI agents with context memory and tool access.



openclaw-node

The Official Bridge

Lightweight client extracted directly from Pinchy UI.

v0.3.1

Protocol v3

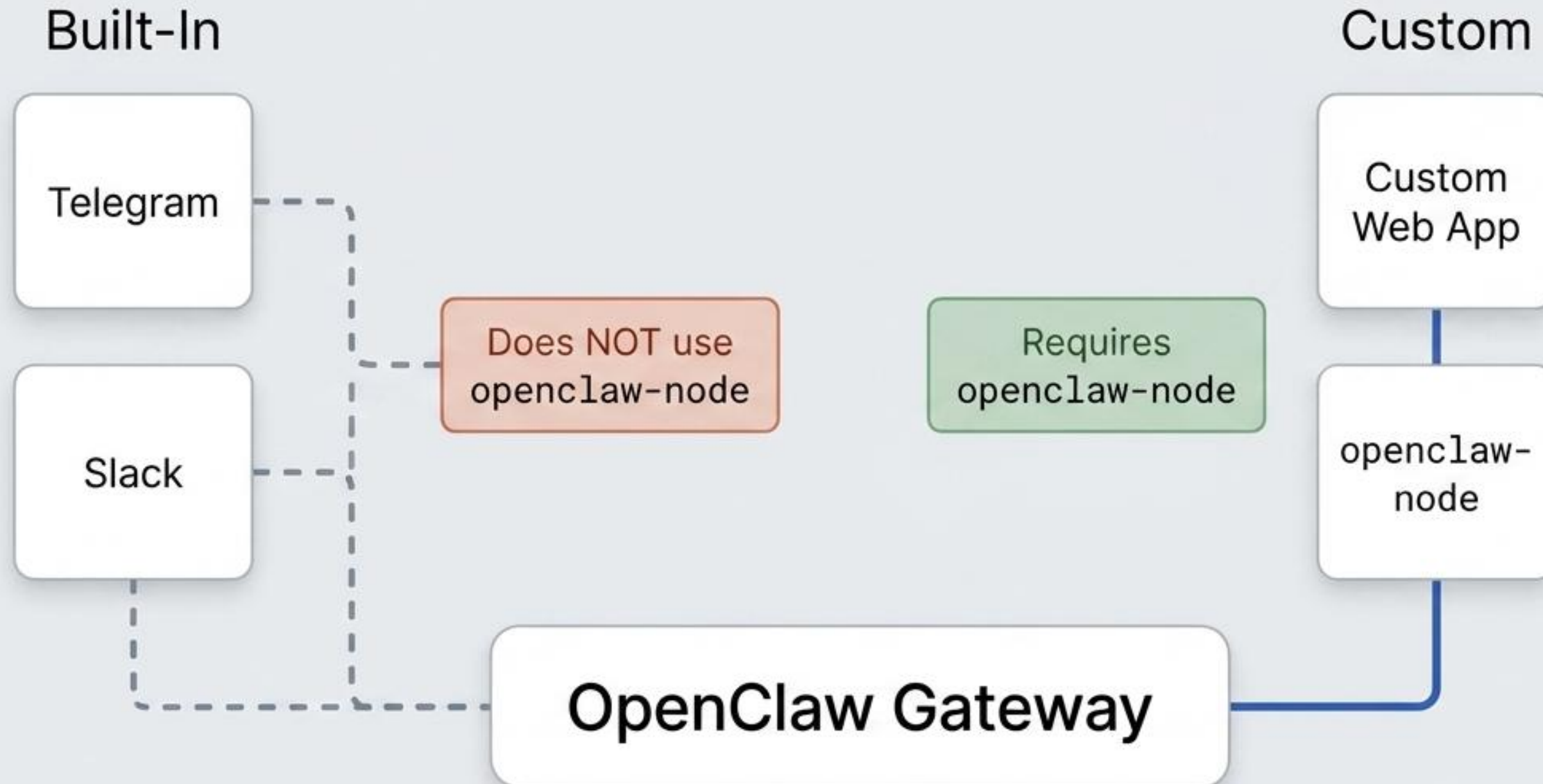
Simple WebSocket vs Openclaw-node Package

Capability	Simple WS Client	openclaw-node
Raw WebSocket transport	✓	✓
Challenge-response auth	✗	✓
Ed25519 device identity	✗	✓
Auto-reconnect / backoff	✗	✓
Req/res correlation + timeouts	✗	✓
Streaming chat iterator	✗	✓
Tool event streaming	✗	✓
Session/config/channel APIs	✗	✓
Typed event system	✗	✓
Message validation & size limits	✗	✓
Chat abort	✗	✓

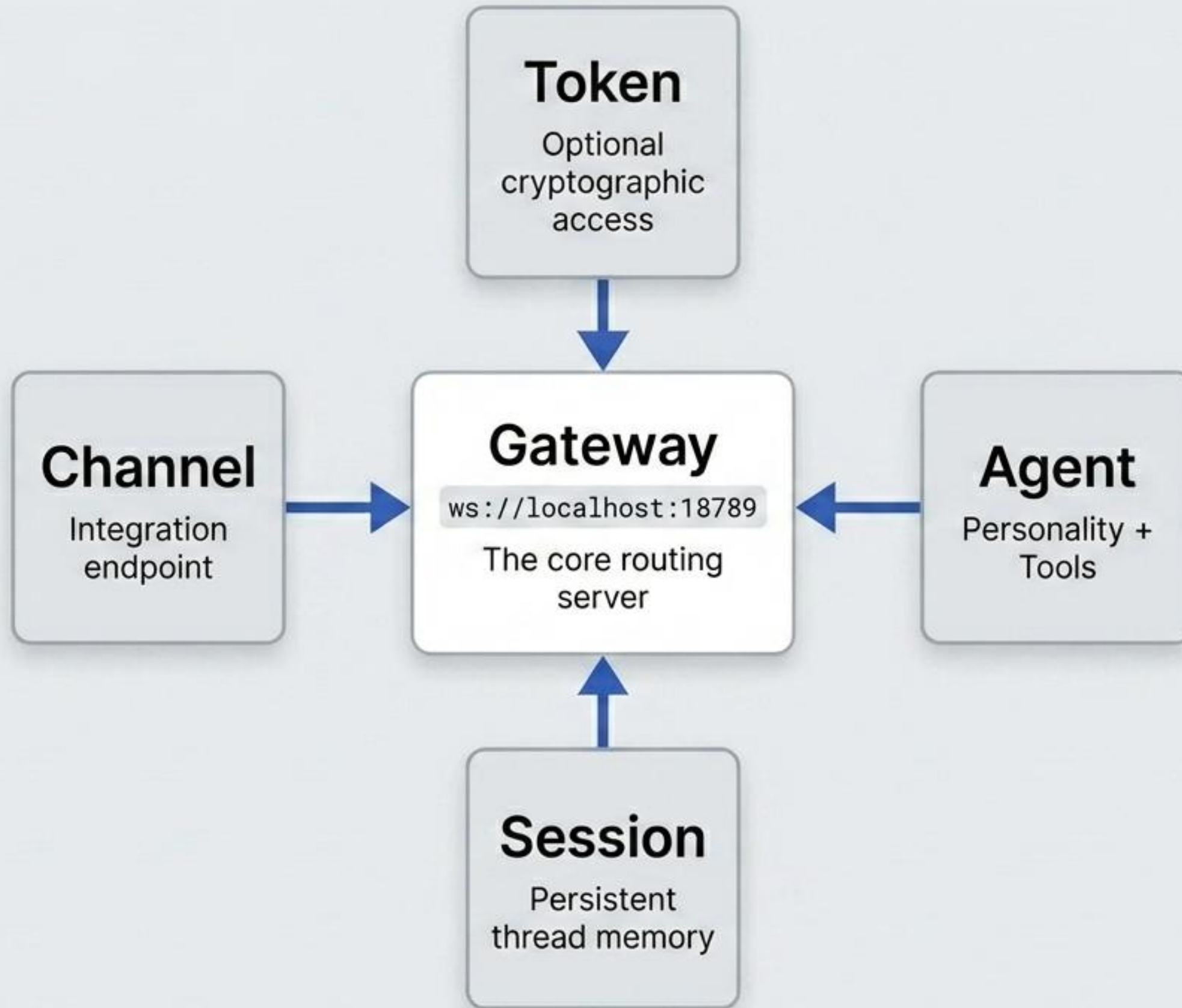
No WebRTC ✗

In short: a simple WebSocket client gives you a pipe; `openclaw-node` gives you a **stateful, resilient, authenticated client with structured RPC, streaming abstractions, and domain-specific APIs** for the OpenClaw ecosystem.

CHANNEL INTEGRATION ARCHITECTURE



Both built-in and custom channels leverage the exact same underlying `WebSocket v3` protocol.



 openclaw-node/

 client.ts (Core logic, 708 lines)

 device.ts (Cryptography, 125 lines)

 types.ts (Protocol, 200 lines)

 index.ts (Public API, 20 lines)



Zero-Config



TypeScript-First



Backwards
Compat



Node 22+
(0 dependencies)



Backwards
Compatible

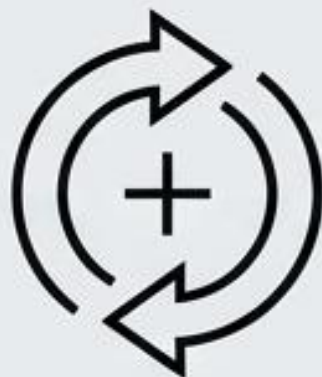


Production-Ready



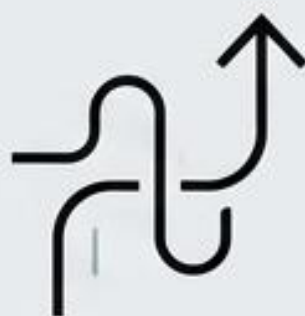
Events

EventEmitter
lifecycle.



RPC

Promise-based
correlation.



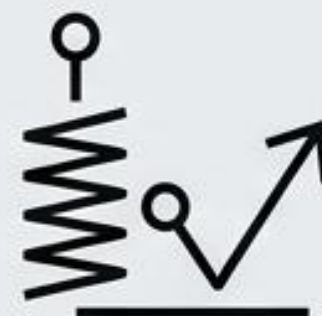
Streaming

AsyncGenerator
chunks.



Identity

Ed25519
Cryptography.



Resilience

Auto-reconnect
backoff.

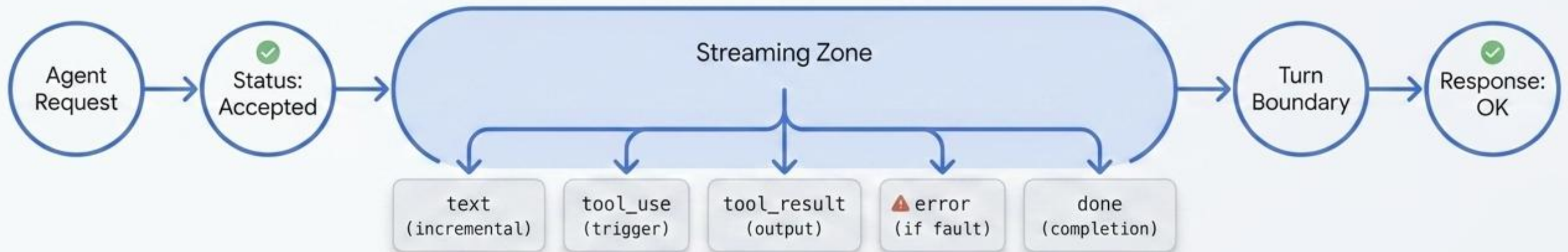
Handshake Flow

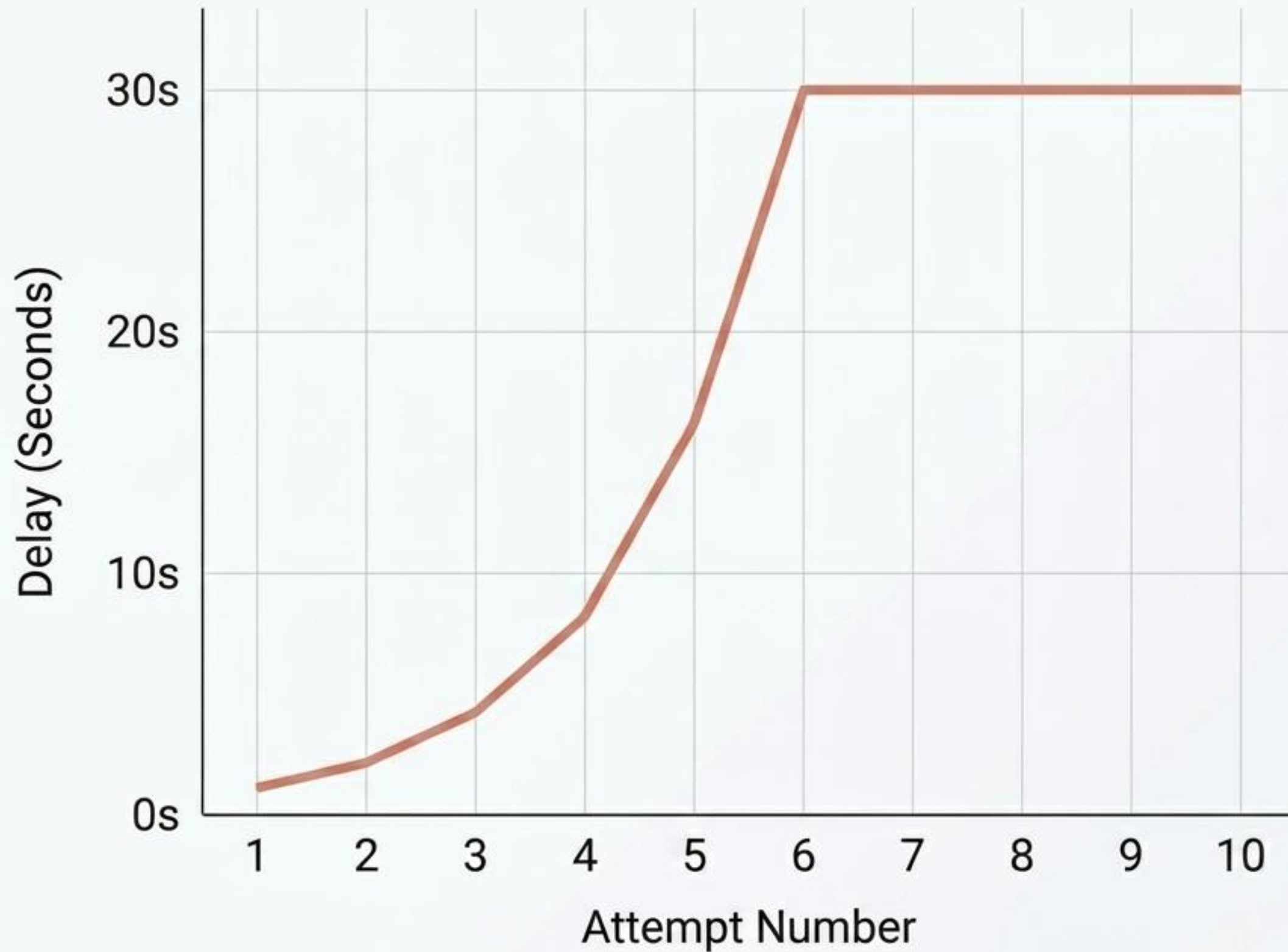


Message Matrix

Type	Description
req	Sends Unique ID, Method, Parameters
res	Returns Matching ID, Success Flag, Payload/Error
event	Server-pushed status updates and notifications

Chat Streaming Lifecycle





Resets counter on
successful connection.

Automatically disabled
upon explicit
`disconnect()` call.

Method	Action	Returns
<code>connect()</code>	Establish connection	<code>Promise<HelloOk></code>
<code>disconnect()</code>	Close connection	<code>Promise<void></code>
<code>chat()</code>	Stream chat	<code>AsyncGenerator</code>
<code>chatSync()</code>	Get full response	<code>Promise<string></code>
<code>chatAbort()</code>	Abort chat	<code>Promise<payload></code>
<code>health()</code>	Check gateway	<code>Promise<payload></code>
<code>request()</code>	Raw RPC	<code>Promise<ProtocolResponse></code>

sessions.*

Manage persistent state

list

history

send

delete

reset

compact

config.*

Handle runtime configuration

get

patch

apply

channels.*

Monitor connection health

status

pairing.*

Handle external auth

list

approve

Transport: wss:// encrypted required

Load: 5MB strict message size limit

Auth: Optional Token authentication

Identity: Ed25519 signing
(Keyfiles 0o600)



Express Backend APIs

Proxying AI requests.



CLI Chatbots

Terminal-based interactions.



Multi-user Web Apps

Custom Pinchy clones.



Custom Messaging

Bridging internal chat tools.

opencLaw-node is the ultimate toolkit for abstracting WebSocket complexities, providing a seamless bridge between your custom infrastructure and the OpenClaw Gateway.

